

Investigating and Implementing LibreRouter and LibreMesh: Virtualized and Raspberry Pi-Based Mesh Networking

Bastas, Dimitrios (SUPREMER LEFTIEUENT COMMANDER)
Kamilos, Dimitrios (SUPREMEISH RIGHTIEUNENT COMMANDER)
Robert Leyba (LT)
Rachel Goldstein (LT)
Margaret Moore (LTJG)
EC3730 Final Project

Abstract—This project investigates LibreRouter and LibreMesh, open-source community networking technologies built on top of OpenWrt. The project explores the architecture, implementation, vulnerabilities, and deployment challenges associated with decentralized mesh networking systems.

A virtualized OpenWrt environment was developed using VMware Workstation Pro to simulate a four-node routed mesh backbone topology. The environment established reproducible experimentation capabilities for distributed routing analysis, virtualization behavior, and future LibreMesh deployment.

The project will later extend the deployment onto Raspberry Pi hardware platforms for physical mesh networking experimentation and wireless performance evaluation.

I. INTRODUCTION

Community mesh networks provide decentralized communication infrastructure capable of operating independently of centralized Internet Service Providers (ISPs). LibreRouter and LibreMesh were developed to support resilient, community-owned wireless infrastructure using open-source software and commodity hardware.

This project investigates the deployment and operation of LibreMesh-based networking systems using both virtualized and physical implementations. The primary goals include evaluating routing behavior, deployment complexity, security considerations, network resilience, and practical limitations of community mesh networking technologies.

In addition to traditional research analysis, this report maintains a reproducible engineering notebook style documenting implementation steps, troubleshooting observations, operational anomalies, and experimental outcomes throughout the project lifecycle.

II. BACKGROUND RESEARCH

A. OpenWrt

OpenWrt is an open-source Linux-based operating system designed primarily for embedded networking devices such as routers and access points. Unlike traditional vendor firmware, OpenWrt provides a writable filesystem, package management system, and modular architecture allowing users to customize routing, firewall, wireless, and network management functionality.

The OpenWrt ecosystem supports x86 virtualized environments in addition to physical embedded hardware, making it suitable for experimental mesh networking research.

B. LibreMesh

LibreMesh is a framework built on top of OpenWrt that simplifies deployment of decentralized mesh networking infrastructure. The platform automates many aspects of wireless mesh configuration while supporting routing protocols such as BATMAN-adv and BMX7.

LibreMesh was designed to support community-operated networks where nodes dynamically discover neighboring devices and establish resilient communication paths without centralized infrastructure.

C. LibreRouter

LibreRouter is an open-hardware networking platform developed specifically for community mesh deployments. The platform combines OpenWrt-based firmware with wireless hardware optimized for long-range and community-managed networking applications.

III. LITERATURE REVIEW AND SECURITY ANALYSIS

A. Identified Weaknesses and Vulnerabilities

Several security concerns and operational limitations were identified during preliminary literature review:

- Firmware update vulnerabilities in OpenWrt package management systems
- Mesh routing attacks including route poisoning and malicious node participation
- Wireless sniffing and unauthorized network access
- Resource exhaustion and denial-of-service attacks
- Complexity of maintaining decentralized routing infrastructure

IV. VIRTUALIZED OPENWRT DEPLOYMENT

A. Virtualization Environment

VMware Workstation Pro 17 was selected as the virtualization platform for the initial implementation phase. Virtualization provided rapid deployment, rollback capability through

snapshots, reproducible experimentation, and simplified troubleshooting prior to deployment on physical Raspberry Pi hardware.

B. OpenWrt Image Preparation

The OpenWrt x86_64 ext4 combined image was downloaded from the official OpenWrt release repository. Since VMware does not natively boot raw IMG files efficiently, the OpenWrt disk image was converted into VMware VMDK format prior to deployment.

C. Virtual Machine Configuration

A custom virtual machine configuration was selected in VMware Workstation Pro to allow manual control over networking interfaces and storage configuration.

TABLE I
INITIAL OPENWRT VM CONFIGURATION

Setting	Value
Guest OS	Other Linux 5.x 64-bit
vCPUs	1
Memory	256 MB
Primary Disk	OpenWrt VMDK
Network Adapter 1	NAT
Network Adapter 2	Host-only

D. Network Interface Design

Two virtual network adapters were configured for each virtual router node:

- NAT adapter for WAN connectivity and package installation
- Host-only adapter for internal mesh backbone communication

This architecture allowed the virtual environment to simulate isolated routed mesh connectivity while preserving Internet access for package installation and updates.

E. Initial Boot Validation

The OpenWrt virtual machine booted successfully after the VMDK disk image was attached to the VMware virtual SCSI controller.

During startup, OpenWrt initialized both virtual Ethernet interfaces successfully:

```
eth0 NIC Link is Up
eth1 NIC Link is Up
```

The successful initialization confirmed that VMware virtual networking devices were correctly recognized by the OpenWrt kernel.



Fig. 1. Successful OpenWrt boot and interface initialization.

V. NETWORK TROUBLESHOOTING AND RECOVERY

A. Initial Connectivity Failure

Initial deployment attempts failed to provide Internet connectivity.

Observed symptoms included:

- Missing default routes
- Failed DNS resolution
- Inability to ping external IP addresses
- DHCP inconsistencies

Example failure:

```
ping -c 4 8.8.8.8
Network unreachable
```

Investigation revealed that VMware host-only networking was unintentionally acting as the WAN DHCP source. Multiple adapter configurations caused interface ambiguity and routing instability.

Observed VMware virtual networks included:

- VMnet1: 192.168.61.0/24
- VMnet8: 192.168.111.0/24

B. Controlled Simplification Strategy

To isolate the networking issue, the second adapter was temporarily removed and the deployment simplified to a single NAT interface.

After simplification, the OpenWrt node successfully received:

- DHCP assignment
- Default gateway
- External DNS access

Working configuration:

```
default via 192.168.111.2 dev eth0
```

Successful validation tests included:

```
ping -c 4 8.8.8.8
ping -c 4 google.com
```

All tests completed successfully with zero packet loss.

C. Package Management Failure Analysis

Package management operations initially failed during APK repository access.

Observed errors included:

```
wget exited with error 1
unexpected end of file
```

Further investigation revealed that BusyBox wget lacked HTTPS support:

```
wget https://downloads.openwrt.org/
HTTPS support not compiled in.
```

This explained repeated failures retrieving APK repository metadata over HTTPS.

The issue represented a significant operational weakness because administrators were effectively forced to downgrade repository transport from HTTPS to HTTP in order to restore package functionality.

Potential security implications included:

- Man-in-the-middle attacks
- Package tampering
- Repository spoofing
- Supply-chain compromise

D. Temporary Package Recovery

HTTPS functionality was partially restored by manually transferring APK packages into the virtual machine and installing them locally:

```
apk add --allow-untrusted ./*.apk
```

Repository transport was later temporarily modified from HTTPS to HTTP to restore repository index access.

After modification:

```
apk update
```

successfully completed.

Core administrative packages were then installed successfully:

```
apk add nano
apk add vim
apk add curl
apk add tcpdump
apk add htop
apk add bash
apk add luci
```

VI. LUCI WEB INTERFACE DEPLOYMENT

After installation of LuCI, the OpenWrt web interface was initially inaccessible through:

```
http://192.168.1.1
```

Diagnostic analysis using:

```
netstat -tlnp
```

confirmed that the web server was actively listening on ports 80 and 443:

```
0.0.0.0:80 LISTEN
0.0.0.0:443 LISTEN
```

Further analysis revealed that the Windows host system could only directly access the VMware NAT interface rather than the internal OpenWrt LAN bridge.

The LuCI interface became accessible using:

```
http://192.168.111.130
```

rather than the internal bridge address.

This demonstrated the distinction between:

- Internal OpenWrt LAN addressing
- Externally reachable VMware NAT addressing
- Hypervisor network segmentation behavior

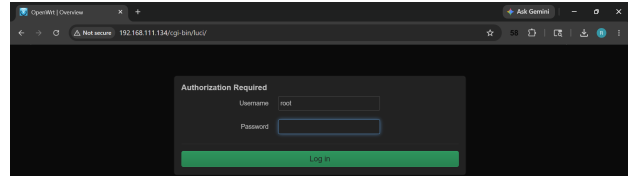


Fig. 2. Accessible LuCI web management login page

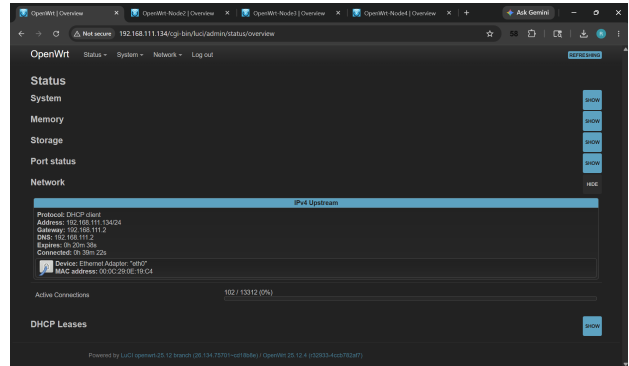


Fig. 3. Accessible LuCI web management interface through VMware NAT networking.

VII. MULTI-NODE MESH BACKBONE CONSTRUCTION

A. Dual-Interface Mesh Backbone

After establishing a stable baseline, a second virtual adapter was reintroduced using VMware host-only networking.

The second interface was assigned as a dedicated mesh backbone network:

```
uci set network.mesh=interface
uci set network.mesh.proto='static'
uci set network.mesh.device='eth1'
uci set network.mesh.ipaddr='10.10.10.1'
uci set network.mesh.netmask='255.255.255.0'
uci commit network
/etc/init.d/network restart
```

Resulting topology:

TABLE II
DUAL-INTERFACE OPENWRT TOPOLOGY

Interface	Address	Purpose
eth0	192.168.111.x	VMware NAT / WAN
eth1	10.10.10.x	Internal mesh backbone

Routing verification confirmed correct segmentation:

```
default via 192.168.111.2 dev eth0
10.10.10.0/24 dev eth1
```

```
root@OpenWrt:~# ip route
default via 192.168.111.2 dev eth0 src 192.168.111.134
10.10.10.0/24 dev eth1 scope link src 10.10.10.1
192.168.1.0/24 dev eth0 scope link src 192.168.1.1
192.168.111.0/24 dev eth0 scope link src 192.168.111.134
root@OpenWrt:~#
```

Fig. 4. Validated OpenWrt Node 1 routing table showing WAN and mesh backbone separation.

B. VM Cloning Strategy

Instead of reinstalling OpenWrt repeatedly, a stable “golden image” virtual machine was cloned using VMware full-clone functionality.

Resulting nodes included:

- OpenWrt-Node1
- OpenWrt-Node2
- OpenWrt-Node3
- OpenWrt-Node4

Each node received:

- Unique hostname
- Unique mesh backbone address
- VMware-generated MAC address

C. Four-Node Mesh Backbone Validation

The resulting mesh backbone addressing scheme included:

TABLE III
FOUR-NODE MESH BACKBONE ADDRESSING

Node	Mesh Address
OpenWrt-Node1	10.10.10.1
OpenWrt-Node2	10.10.10.2
OpenWrt-Node3	10.10.10.3
OpenWrt-Node4	10.10.10.4

Connectivity testing was performed between all nodes using ICMP echo requests.

Successful bidirectional communication was verified between all nodes with:

- Zero packet loss
- Stable latency
- Successful route propagation

The successful validation demonstrated:

- Stable Layer 2 adjacency
- Functional Layer 3 routing
- Operational VMware host-only backbone networking
- Successful multi-node OpenWrt virtualization

```
root@OpenWrt-Node4:~# ping -c 2 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data:
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=2.18 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=1.33 ms

--- 10.10.10.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1004ms
rtt min/avg/max/mdev = 1.334/1.755/2.176/0.421 ms
root@OpenWrt-Node4:~# ping -c 2 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data:
64 bytes from 10.10.10.2: icmp_seq=1 ttl=64 time=2.23 ms
64 bytes from 10.10.10.2: icmp_seq=2 ttl=64 time=4.85 ms

--- 10.10.10.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 2.234/3.542/4.851/1.308 ms
root@OpenWrt-Node4:~# ping -c 2 10.10.10.3
PING 10.10.10.3 (10.10.10.3) 56(84) bytes of data:
64 bytes from 10.10.10.3: icmp_seq=1 ttl=64 time=1.92 ms
64 bytes from 10.10.10.3: icmp_seq=2 ttl=64 time=1.48 ms

--- 10.10.10.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 1.480/1.700/1.921/0.220 ms
root@OpenWrt-Node4:~#
```

Fig. 5. Successful four-node mesh backbone connectivity validation.

D. Sequential Startup and DHCP Convergence

Initial simultaneous startup of cloned nodes produced temporary WAN IP duplication behavior within the VMware NAT network.

Multiple nodes briefly inherited identical DHCP leases from previous cloned states.

This behavior was mitigated by sequential node startup. Sequential startup allowed VMware DHCP services to converge properly and assign unique WAN addresses based on MAC address differentiation.

Final WAN assignments stabilized as:

- Node1: 192.168.111.134
- Node2: 192.168.111.136
- Node3: 192.168.111.137
- Node4: 192.168.111.138

This demonstrated:

- DHCP lease convergence behavior
- Temporary ARP instability during cloned startup
- Virtualization timing dependencies
- Successful MAC-based lease differentiation

E. IPv6 Duplicate Address Detection Behavior

During sequential startup of cloned nodes, OpenWrt repeatedly reported IPv6 duplicate address detection warnings:

IPv6 duplicate address detected

Despite repeated IPv6 duplicate detection events:

- IPv4 routing remained stable
- Mesh backbone communication remained operational
- Node-to-node connectivity was unaffected

This behavior represents an important operational consideration for cloned embedded router deployments and virtualized distributed networking systems.

```

[ 17.533917] procd: - init -
Please press Enter to activate this console.
[ 17.759256] kmodloader: loading kernel modules from /etc/modules.d/*
[ 17.790095] urngd: v1.0.2 started.
[ 17.851278] e1000: Intel(R) PRO/1000 Network Driver
[ 17.859459] e1000: Copyright (c) 1999-2006 Intel Corporation.
[ 18.500979] e1000 0000:02:00:0 eth0: (PCI:32MHz:32-bit) 00:0c:29:8a:68:7e
[ 18.509535] e1000 0000:02:00:0 eth0: Intel(R) PRO/1000 Network Connection
[ 19.141860] e1000 0000:02:01:0 eth1: (PCI:66MHz:32-bit) 00:0c:29:8a:68:80
[ 19.147917] e1000 0000:02:01:0 eth1: Intel(R) PRO/1000 Network Connection
[ 19.165779] ixgbe: Intel(R) 10 Gigabit PCI Express Network Driver
[ 19.172232] ixgbe: Copyright (c) 1999-2016 Intel Corporation.
[ 19.185750] i2c dev: i2c /dev entries driver
[ 19.237306] PPP generic driver version 2.4.2
[ 19.244786] NET: Registered PF_PPPOX protocol family
[ 19.364798] kmodloader: done loading kernel modules from /etc/modules.d/*
[ 21.753483] 8021q: adding VLAN 0 to HW filter on device eth0
[ 21.764898] e1000: eth0 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: N
one
[ 22.005832] 8021q: adding VLAN 0 to HW filter on device eth1
[ 22.013822] e1000: eth1 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: N
one
[ 22.886777] IPv6: eth0: IPv6 duplicate address fdd:b689:d48c::1 used by 08:0
c:29:8a:19:c4 detected!

```

Fig. 6. Observed IPv6 duplicate address detection during sequential startup of cloned OpenWrt nodes.

VIII. FUTURE LIBREMESH AND BATMAN-ADV DEPLOYMENT

Future implementation phases will include:

- Installing LibreMesh packages
- Configuring BATMAN-adv mesh routing
- Constructing multi-hop routing experiments
- Measuring convergence behavior
- Evaluating node-failure resilience
- Testing route propagation across distributed nodes

IX. FUTURE RASPBERRY PI DEPLOYMENT

The final implementation stage will migrate the virtualized mesh environment onto Raspberry Pi hardware devices running OpenWrt and LibreMesh.

The Raspberry Pi deployment will evaluate:

- Physical wireless mesh formation
- Real-world throughput and latency
- Hardware limitations
- Deployment obstacles
- Inter-group node interoperability

X. CONCLUSION

The virtualization phase successfully established a functional four-node OpenWrt distributed routing environment capable of supporting future LibreMesh experimentation.

The implementation demonstrated:

- Successful OpenWrt virtualization
- Stable multi-node routed mesh backbone construction
- Functional distributed node communication
- Operational DHCP convergence behavior
- Reproducible deployment methodology
- Identification of virtualization and IPv6 edge cases

The completed environment now represents a reproducible distributed networking research platform suitable for future dynamic mesh routing experimentation using LibreMesh and BATMAN-adv.

REFERENCES

- [1] OpenWrt Project, "OpenWrt Documentation," [Online]. Available: <https://openwrt.org>
- [2] LibreMesh Project, "LibreMesh Documentation," [Online]. Available: <https://libremesh.org>
- [3] BMX7 Routing Protocol, "BMX7 GitHub Repository," [Online]. Available: <https://github.com/bmx-routing/bmx7>